Offsets

- Register offset

```
LDR r1, [r0, r3] ; offset = r3
LDR r1, [r0, r3, LSL #2] ; offset = r3*4
```

- Immediate offset

```
Pre-index: LDR r1, [r0, #4] ; r0+4 (updates offset before)
Post-index: LDR r1, [r0], #4 (updates offset after)
Pre-index w update: LDR r1, [r0, #4]!
```

Store Multiple Registers

```
STMxx r0! {r3, r1, r7, r2}
```

Subroutines

R0-R12: general purpose
R13: stack pointer
R14: link register
R15: program counter

R0-R3: caller saved (LR) R4-R11: callee saved +(R14) +(R13)

PUSH: decrement before POP: increment after

Interrupts

FPU was not used before int

LR	Return Stack
...F1	MSP
...F9	MSP
...FD	PSP

FPU was used before int.

LR	Return Stack
.....E1	MSP
.....E9	MSP
.....ED	PSP

```
SysTick_Handler
  EXPORT ...
  TST LR, #4          ; check last 2 of LR
  MRSEQ r0, MSP       ; if 0 MSP
  MRSNE r0, PSP
  STMFD r0!, {r7-r9}  ; push onto full-descending stack
```

NVIC_DisableIRQ(IRQn) NVIC_EnableIRQ(IRQn) NVIC_ClearingPendingIRQ() NVIC_SetPriority(IRQn, priority)

- Disable / Enable Peripheral Interrupts

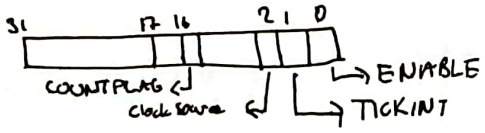
```
NVIC->ISER[IRQn >> 5] = 1 << (IRQn & 0x1F); //enable
```

- Auto-Stacking

r0, r1, r2, r3, r12, LR(r14), PC(r15), xPSR

Systick

SysTick_CTRL




```
void SysTick_Initialize (uint32_t ticks)
  SysTick->CTRL = 0; // disable systick
  SysTick->LOAD = ticks-1; // set reload
  NVIC_SetPriority (SysTick_IRQn, (1<<__NVIC_PRIO_BITS)-1); // least urgency
  SysTick->VAL = 0; // reset the counter
  SysTick->CTRL |= SysTick_CTRL_CLKSOURCE // select processor clock
  SysTick->CTRL |= SysTick_CTRL_TICKINT; // enable systick interrupt
  SysTick->CTRL |= SysTick_CTRL_ENABLE; // enable systick
```

- Calculating reload value

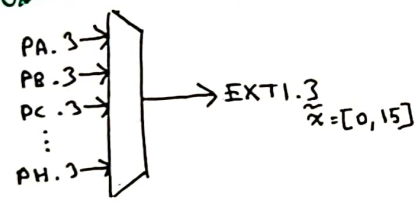
Suppose clock source = 80MHz Goal: Systick interval 10ms

$$Reload = \frac{10ms}{Clock\ period} - 1 = \underbrace{10ms}_{10\times10^{-3}} \times \underbrace{clock\ freq}_{80\times10^{6}} * 1 = 799999$$

Interval = (RELOAD+1) x Source_Clock_Period

Systick int period = (1 + SysTick_LOAD) x $\frac{1}{f_{CLK_SYT_COUNTER}}$

External Interrupts



Interrupt Number (8-bits)	Memory Addr of ISR (32 bits)
EXTI0_IRQn = 6	EXTI0_IRQHandler
...	...
EXTI9_5_IRQn = 23	
...	
EXTI15_10_IRQn = 40	

SYSCFG_EXTICR{0-4}_EXTI{0-15}_P{A-G}

0 → 0,1,2,3 → A-G

1 → 4,5,6,7 → A-G

...

GPIO

- Set pin A.14 to high

In C

```
GPIOA->ODR |= 1UL<<14
```

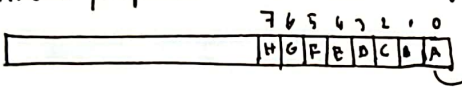
In Assembly

```
GPIOA_BASE_ EQU ...
GPIO_ODR    EQU 0x14

LDR r7, =GPIOA_BASE
LDR r1, [r7, #GPIO_ODR]
ORR r1, r1, #(1<<14)
STR r1, [r7, #GPIO_ODR]
```

- Enable clock

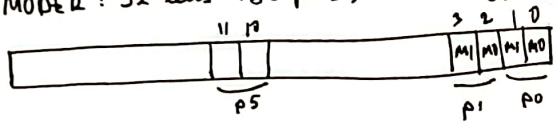
AHB2 peripheral clock enable register (RCC_AHB2ENR)




```
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN
```

- GPIO mode

MODER : 32 bits (16 pins) 00: input 01: output



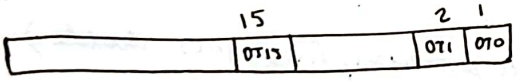

```
GPIOA->MODER &= ~(3UL<<10); // clear bits 11 and 10 for pin 5
GPIOA->MODER |= 1UL<<10;    // set bit 10, set pin 5 as output
```

- Output type

```
GPIOA->OTYPE &= ~(1UL<<5);  // clear bit 5
GPIOA->OTYPE &= ~(GPIO_OTYPER_OT_5); // clear bit 5
```

in the header → `#define GPIO_OTYPER_OT_5 = 0x00000020`

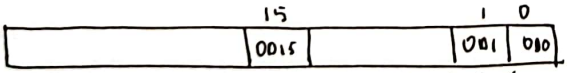
OTYPE : 16 bits reserved, 16 bits for data, 1 bit for each pin



0: push-pull 1: open-drain

- Output data

ODR: 16 bits reserved, 16 data bits, 1 bit for each pin

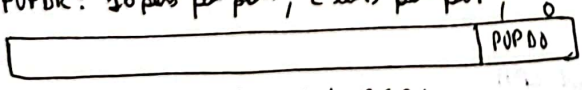



```
GPIOA->ODR |= 1UL<<5; // set bit 5
```

`GPIOA->ODR |= GPIO_ODR_OR_5;`

- Pull-up, Pull-down

PUPDR: 16 pins per port, 2 bits per pin

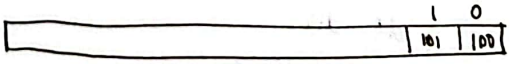


00: no pull-up / pull-down
01: pull up

```
GPIOC->PUPDR &= ~3UL<<26;
GPIOC->PUPDR &= ~GPIO_PUPDR_PUPDR13;
```

- Input data

IDR: 16 bits reserved, 16 data bits (1 bit per pin)




```
uint32_t input = (GPIOA->IDR & GPIO_IDR_IDR_13) >> 13;
```

or

```
uint32_t mask = 1UL<<13; uint32_t input = (GPIOC->IDR & mask) == mask
```

Fixed-Point Numbers

$10101.101_2 = \frac{10101101}{2^3} = 21.625$ UQ5.3

- Convert float to fixed point

$f = 3.141593$ to UQ4.12 (Q → two's complement)

- $f \times 2^{12} = 12867.964928$
- Round 12868
- Convert to binary $11_0010_0100_0100_2$
- Organize into UQ4.12: $0011.0010_0100_0100_2$
- Final result is hex: 0x3244
- Error: $\frac{12868}{2^{12}} - f = -8.5625 \times 10^{-6}$

- Why fixed point

Lower cost, more bits for precision, higher performance

Floating point: wide dynamic range, shorter development time

Floating-point Numbers

- Normalization

$\underbrace{10.746 \times 2^6}_{\text{original}} = \frac{10.746}{8} \times 8 \times 2^6 = 1.34325 \times 2^9$

- IEEE 754

$(-1)^S \times (1 + \text{fraction}) \times 2^{\text{exponent - bias}}$ (bias = 127 single precision, bias = 1023 double)

Floating-point Unit

- Enable FPU

```
SCB->CPACR |= (0xF << 20);
LDR r0, =SCB_BASE
LDR r1, [r0, #SCB_CPACR]
ORR r1, r1, #(0xF << 20) ; enable full access to CP10 and CP11
STR r1, [r0, #SCB_CPACR]
```

00: access denied 01: privileged access 10: reserved 11: full access

Registers: s0-s31, d0-d15

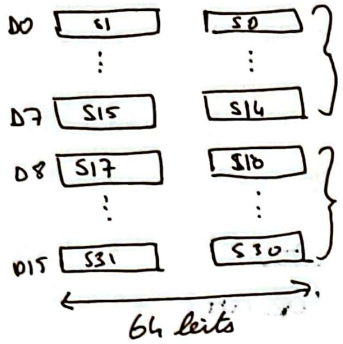
VLDR.F32 s0, [r0] load float

VLDR.F64 d0, [r0] load double

```
VMOV s0, s1, r0, r1  ; s0 = r0, s1 = r1
VMOV r1, d0 1        ; r1 = d0 [1] = s1
VMOV.F32 s1, #3.25   ; s1 = 3.25
VMRS APSR_nzcv, FPSCR ; copy NZCV flags to APSR
```

FPU special registers
FPSCR, FPCAR, FPCCR

Only CMP and CMPF update N, Z, C, V flags

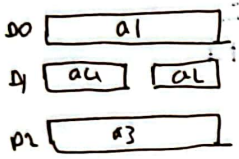


caller saved (VPUSH / VPOP)

caller saved

- Passing arguments

double fun (double a1, float a2, double a3, float a4)

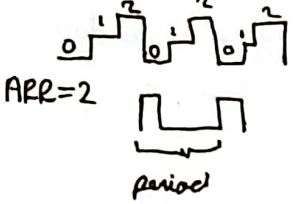



```
area PROC
EXPORT area
VLDR.F32 s1, #3.1416
VMUL.F32 s1, s1, s0
VMUL.F32 s0, s1, s0
BXLR
ENDP
```

16/06/2023

General Purpose Timers

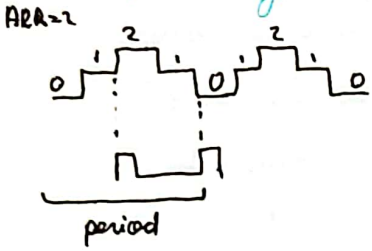
– Edge aligned mode



Period = (1+ARR) * clock period $\longrightarrow \frac{(ARR+1)(PSC+1)}{f_{source}}$

Clock period = $\frac{1}{f_{CNT}}$ $\quad f_{CNT} = \frac{f_{source}}{1+PSC}$

– Center aligned mode



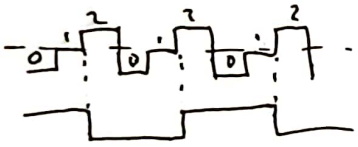
Period = 2 * ARR * clock period

UEV period = (1+ARR) × (1+Prescaler) × $\frac{1}{f_{CK_CNT}}$

– PWM mode 1 (low-true)

Timer output (OCREF) = $\begin{cases} \text{high if counter} < CCR \\ \text{low if counter} \geq CCR \end{cases}$ $\quad$ duty cycle = $\frac{CCR}{ARR+1}$

ARR=2, CCR=1



mode 2 (high-true) $\quad$ duty cycle = $1 - \frac{CCR}{ARR+1}$

mode 3 (center aligned) $\quad$ duty cycle = $1 - \frac{CCR}{ARR}$

– Input Capture

t_1: hw copies counter to CCR, generates an interrupt to run timer ISR (first rising edge)

t_2: same as t_1 (second rising edge)

time span = $|CCR_{new} - CCR_{old}| \times \frac{1}{f_{CL_CNT}}$ $\quad f_{CL_CNT} = \frac{f_{CL_PSC}}{1+PSC}$

- Configuring SysTick in assembly

```
SysTick_Initialize PROC
  EXPORT SysTick_Initialize
  LDR r0, =SysTick_BASE
  MOV r1, #0
  STR r1, [r0, #SysTick_CTRL]      } disable systick IRQ and counter, select external clock
  LDR r2, =262
  STR r2, [r0, #SysTick_LOAD]      } specify the number of cycles between interrupts
  MOV r1, #0
  STR r1, [r0, #SysTick_VAL]       } clear systick current value
  LDR r2, =SCB_BASE
  ADD r2, r2, #SCB_SHP
  MOV r3, #1<<4
  STRB r3, [r2, #11]               } set priority as 1
  LDR r1, [r0, #SysTick_CTRL]
  ORR r1, r1, #3
  STR r1, [r0, #SysTick_CTRL]      } enable systick counter and interrupt
  BX lr
  ENP
```

1. Disable
2. Set reload value
3. Set current val to zero
4. Set clock source
5. Enable tickint
6. Enable systick
7. Set int. priority
8. Enable interrupt

```
void SysTick_Init(uint32_t ticks)
  SysTick->CTRL = 0;
  SysTick->LOAD = ticks-1;
  NVIC_SetPriority(Systick_IRQn, (1<<__NVIC_PRIO_BITS)-1);
  SysTick->VAL = 0;
  SysTick->CTRL |= SysTick_CTRL_CLKSOURCE_Msk; // select the processor clock
  SysTick->CTRL |= SysTick_CTRL_TICKINT_Msk;   //
  SysTick->CTRL |= SysTick_CTRL_ENABLE_Msk;
  NVIC_EnableIRQ(SysTick_IRQn)
```

- Delay

```
delay PROC
  EXPORT delay
  ;r0 = delay input
  MOV r10, r0 ; make a copy of time delay
loop CMP r10, #0
  BNE loop
  BX lr
  ENDP
```

- GPIO Output

1. Enable clock 2. Enable mode 3. Set output type 4. Set output speed 5. Set pull-up / pull-down

6. Set output data register

- B.2 output high

```
LDR r2, =RCC_BASE
LDR r1, [r2, #RCC_AHB2ENR]
ORR r1, r1, 2                ; set bit 2 of AHB2ENR
STR r1, [r2, #RCC_AHB2ENR]   ; port B clock enable

LDR r3, =GPIOB_BASE          ; GPIO port B base addr

LDR r1, [r3, #GPIO_MODER]    ; read the mode register
BIC r1, r1, #(3<<4)          ; direction mask pin 6, clear bits 5 and 4
ORR r1, r1, #(1<<4)          ; set mode as digital output (01)
STR r1, [r3, #GPIO_MODER]    ; save the mode register

LDR r1, [r3, #GPIO_OTYPER]   } set pin 2 push-pull mode
BIC r1, r1, #(1<<2)          }
STR r1, [r3, #GPIO_OTYPER]   }

LDR r1, [r3, #GPIO_OSPEEDR]  } set output speed as low
BIC r1, r1, #(3<<4)          }
STR r1, [r3, #GPIO_OSPEEDR]  }

LDR r1, [r3, #GPIO_PUPDR]    } set 11 as no pull-up, no pull-down
BIC r1, r1, #(3<<4)          }
STR r1, [r3, #GPIO_PUPDR]    }

LDR r1, [r3, #GPIO_ODR]      } set bit 2 (light up LED)
ORR r1, r1, #(1<<2)          }
STR r1, [r3, #GPIO_ODR]      }
```

- A.13

```
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN
GPIOA->MODER &= ~(3UL<<2*LED_PIN)   // clear mode bits
GPIOA->MODER |= 1UL<<(2*LED_PIN)    // set mode to output (01)
GPIOA->OSPEEDR &= ~(3UL<<2*LED_PIN) // clear output speed bits (00) (slow)
GPIOA->OTYPER &= ~(1<<LED_PIN)      // push-pull output
GPIOA->PUPDR &= ~(3<<2*LED_PIN)     // no pull-up, no pull-down
```

GPIO input and external interrupt

```
RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOCEN;          } enable clocks

SYSCFG->EXTICR[0] &= ~SYSCFG_EXTICR4_EXTI13;
SYSCFG->EXTICR[0] |= SYSCFG_EXTICR4_EXTI13_PC; } select PC.13 as trigger source

EXTI->RTSR1  |= EXTI_RTSR1_RT13; // enable rising edge trigger

EXTI->FTSR1  &= ~EXTI_FTSR1_FT13; // disable falling edge trigge

EXTI->IMR1   |= EXTI_IMR1_IM13;   // enable extr 13 interrupt

NVIC_SetPriority(EXTI15_10_IRQn, 1);
NVIC_EnableIRQ(EXTI15_10_IRQn)
GPIOC->MODER  &= ~(3UL<<(2*BTN_PIN));  // clear mode bits (00) = input
GPIOC->PUPDR  &= ~(3UL<<(2*BTN_PIN));  // no pull-up, no pull-down
```

1. Enable clock
2. Choose trigger
3. Set mode
4. Set pu/pd

Interrupt Handler

```
if ((EXTI->PR1 & EXTI_PR1_PIF13) == EXTI_PR1_PIF13){   // != 0
  ...
  EXTI->PR1 |= EXTI_PR1_PIF13; // clear pending
}
```

1. check int. pending flag
2. clear pending

```
if (GPIOA->IDR &01 == 0x1)      // check input on pin PA.0
  ...
if (GPIOC->IDR & mask == mask)
```